

EX.NO:5 – ADT Queue Using Exception Handling

Program

```
// QueueADT.java

interface QueueADT<T> {

    void enqueue(T item) throws QueueOverflowException;

    T dequeue() throws QueueUnderflowException;

    boolean isEmpty();

    boolean isFull();

}

// Custom Exceptions

class QueueOverflowException extends Exception {

    public QueueOverflowException(String message) {

        super(message);

    }

}

class QueueUnderflowException extends Exception {

    public QueueUnderflowException(String message) {

        super(message);

    }

}

// CircularQueue.java

class CircularQueue<T> implements QueueADT<T> {

    private T[] queue;

    private int front, rear, size, capacity;

    @SuppressWarnings("unchecked")

    public CircularQueue(int capacity) {
```

```
    this.capacity = capacity;

    queue = (T[]) new Object[capacity];

    front = rear = size = 0;
}
```

```
public void enqueue(T item) throws QueueOverflowException {
    if (isFull()) throw new QueueOverflowException("Queue Overflow!");

    queue[rear] = item;

    rear = (rear + 1) % capacity;

    size++;
}
```

```
public T dequeue() throws QueueUnderflowException {
    if (isEmpty()) throw new QueueUnderflowException("Queue Underflow!");

    T item = queue[front];

    front = (front + 1) % capacity;

    size--;

    return item;
}
```

```
public boolean isEmpty() { return size == 0; }

public boolean isFull() { return size == capacity; }

}
```

```
// Main.java
```

```
public class Main {

    public static void main(String[] args) {

        try {

            CircularQueue<Integer> q = new CircularQueue<>(3);

            q.enqueue(10);

            q.enqueue(20);

        }
    }
}
```

```
        q.enqueue(30);  
        System.out.println("Dequeued: " + q.dequeue());  
        q.enqueue(40);  
    } catch (Exception e) {  
        System.out.println(e.getMessage());  
    }  
}  
}
```

s

Output

Dequeued: 10

Queue Overflow!